Bash Shell Scripting

Session 1 — Beginner's Guide

What is Bash?

Bash is a scripting language that lets you talk to your computer and automate tasks. Instead of clicking buttons, you write commands — and the computer follows them in order.

- Bash stands for Bourne Again SHell
- It runs on Linux, macOS, and Windows (via WSL)
- It's used for automation, file management, system administration

1.1 Your First Script

Step 1 — Create a new file:

```
nano hello.sh
```

Step 2 — Write this inside the file:

```
#!/bin/bash

echo "Hello, World!"
```

> 💡 *The first line #!/bin/bash is called the "shebang". It tells the computer to use Bash to run this file. Always start your scripts with it.*

Step 3 — Make it executable and run it:

```
chmod +x hello.sh
./hello.sh
```

1.2 Variables

Variables store information you can use later. No spaces around the = sign!

```
#!/bin/bash

NAME="Ahmed"
AGE=20

echo "My name is $NAME"
echo "I am $AGE years old"
echo "Hello ${NAME}!"   # curly braces useful when adding text around
variable
```

⚠ **NAME = "Ahmed" is WRONG — no spaces around =**

⚠ **NAME="Ahmed" is CORRECT**

Getting input from the user with read:

```
#!/bin/bash
echo "What is your name?"
read NAME
echo "Hello, $NAME! Welcome to Bash."
```

1.3 Comments

```
# This is a single line comment

: '
This is
a multi-line
comment
'
```

Solution:

```
#!/bin/bash
echo "Enter your name:"
```

```
read NAME
echo "Enter your city:"
read CITY
echo "Hello $NAME, you are from $CITY!"
```

2.1 if / elif / else

Conditionals let your script make decisions. The basic structure is:

```
if [[ condition ]]; then
    # do something
elif [[ other condition ]]; then
    # do something else
else
    # default action
fi
```

Example 1 — Number comparison:

```
#!/bin/bash
echo "Enter a number:"
read NUM

if [[ $NUM -gt 10 ]]; then
    echo "Greater than 10"
elif [[ $NUM -eq 10 ]]; then
    echo "Equal to 10"
else
    echo "Less than 10"
fi
```

Example 2 — String check:

```
#!/bin/bash
echo "Enter your name:"
read NAME
Read 20
if [[ $NAME == "Ahmed" && $AGE == 20 ]]; then
    echo "Welcome back, Ahmed!"
else
    echo "Hello, $NAME! You are new here."
fi
```

Example 3 — Check if a file exists:

```
#!/bin/bash
if [[ -f "hello.sh" ]]; then
    echo "File exists!"
else
    echo "File not found."
fi
```

2.2 Comparison Operators Reference

Operator / Syntax	Meaning
`-eq`	Equal (numbers): $a -eq $b
`-ne`	Not equal (numbers)
`-gt`	Greater than
`-lt`	Less than
`-ge`	Greater than or equal
`-le`	Less than or equal
`==`	Equal (strings)
`!=`	Not equal (strings)
`-f FILE`	File exists
`-d DIR`	Directory exists

☐ Mini Exercise 2

☐ **Exercise 2 — Grade checker script:**

1. Ask the student for their grade (0–100)
2. Print "Pass" if grade is 50 or above
3. Print "Fail" if below 50
4. Bonus: Print "Excellent!" for grades 90 and above

Solution:

```
#!/bin/bash
echo "Enter your grade:"
read GRADE
```

```
if [[ $GRADE -ge 90 ]]; then
    echo "Excellent!"
elif [[ $GRADE -ge 50 ]]; then
    echo "Pass"
else
    echo "Fail"
fi
```

3.1 For Loop

Basic range:

```
for i in {1..5}; do
    echo "Number: $i"
done
```

Loop over a list:

```
for NAME in Ahmed Sara Omar; do
    echo "Hello, $NAME!"
done
```

With step (count by 2s):

```
for i in {2..10..2}; do
    echo $i
done
```

3.2 While Loop

```
#!/bin/bash
COUNT=1

while [[ $COUNT -le 5 ]]; do
    echo "Count is: $COUNT"
    ((COUNT++)) = count =2
                  new-count = count+1
done
```

💡 *Use for when you know how many times to repeat. Use while when you want to keep going until something changes.*

☐ Mini Exercise 3

☐ **Exercise 3 — Multiplication table:**

5. Write a loop that prints the multiplication table of 3
6. Go from 3x1 up to 3x10

Solution:

```
#!/bin/bash
for i in {1..10}; do
    echo "3 x $i = $((3 * i))"
done
```

3.3 Functions

Functions let you reuse code. Define once, call many times.

```
#!/bin/bash

greet() {
    echo "Hello, $1!"
}

greet "Ahmed"
greet "Sara"
```

💡 *$1 inside a function means the first argument passed to it. $2 would be the second, and so on.*

Function with logic:

```
#!/bin/bash

check_age() {
    if [[ $1 -ge 18 ]]; then
        echo "$1 is an adult"
    else
        echo "$1 is a minor"
```

```
    fi
}

check_age 20
check_age 15
```

Final Exercise — Put it all together!

Final Exercise — Even or Odd checker:

7. Write a function called is_even that checks if a number is even or odd
8. Use a loop to check numbers from 1 to 10
9. Print "[number] is even" or "[number] is odd" for each

Solution:

```
#!/bin/bash

is_even() {
    if (( $1 % 2 == 0 )); then
        echo "$1 is even"
    else
        echo "$1 is odd"
    fi
}

for i in {1..10}; do
    is_even $i
done
```

Session Summary

Topic	Key Syntax	Key Takeaway
Variables	`NAME="Ahmed"`	No spaces around =
User Input	`read NAME`	Gets input from the user
if / else	`if [[ condition ]]; then`	Use -gt, -eq, == for comparisons
for loop	`for i in {1..5}; do`	Best for known ranges
while loop	`while [[ condition ]]; do`	Best when count is unknown
Functions	`myfunc() { ... }`	Use $1 $2 for arguments

Happy Scripting!